PES UNIVERSITY

Department of Computer Science and Engineering

UE24CS326: COMPUTER NETWORK SECURITY · ASSIGNMENT 1

Packet Sniffing & Spoofing using Python

STUDENT

Sumedh Girish

INSTRUCTOR

Dr. Preet Kanwal

DATE

August 15, 2026

Setup

The Lab gives a docker compose configuration, which we have to start before any programs can we run. This should give us 3 machines, an `attacker`, `HostA` and `HostB`. This can be done by running

```
docker compose up
```

in the `Labsetup` directory.

We begin by installing scapy, which is a python library that can be used to sniff and spoof packets over networks easily. This is done by importing the library as follows.

```
from scapy.all import *
```

Next, we have to choose the Network Interface Card (NIC) we are working with. For this lab, since the program executes inside an attacker docker container, we reference the **bridge interface** of the attacker machine.

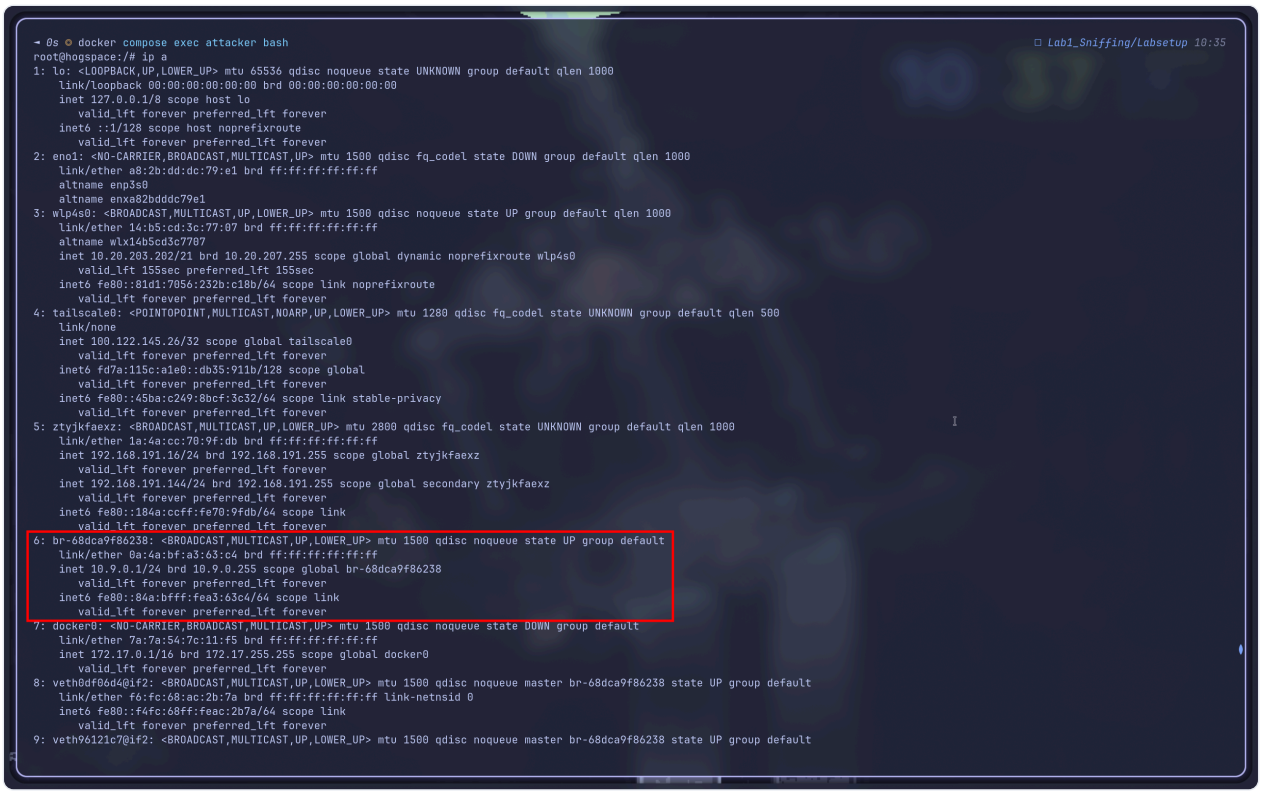

Figure 1: Finding attacker NIC

```
BRIDGE_INTERFACE = "br-68dca9f86238"
```

Finally, to segregate between subsection solutions, we create a class called `Task1` which will contain all code for this assignment.

Question 1 · Sniffing Packets

The objective of this task is to learn how to use Scapy to do packet sniffing in Python programs.

SOLUTION

Task 1A

```
@staticmethod
def _1A():
     def capture(pkt):
          print(pkt.summary(), flush=True)

     _ = sniff(iface=BRIDGE_INTERFACE, prn=capture)
```

This code tries to print all packets it sees over the interface. This is done using the `sniff(...)` function exposed by the scapy library. The `iface` parameter refers to the target NIC and the `prn` parameter allows specifying a callback function which is called every time a packet arrives at the interface.

Task 1B

Next we look towards segregating the output to only packets we are interested in. While this can be written with vanilla python code, the magnitude of packets can make the program slow. Thus scapy allows implicitly performing this operation using **packet filters**.

These packet filters are compiled and applied by the kernel directly onto the NIC driver configuration — making the filtering extremely efficient.

```
@staticmethod
def _1B():
     def capture(pkt):
          print(pkt.summary(), flush=True)

     filterA = "icmp"
     filterB = "tcp and src host 10.9.0.5 and dst port 23"
     filterC = "net 10.9.0.0/16"

     _ = sniff(iface=BRIDGE_INTERFACE, filter=filterA, prn=capture)
```

The above code demonstrates a few different types of filters in action. We apply filters by passing the packet filter code as a string into the `filter` option in `sniff(...)`.

1. The `filterA` accepts all ICMP packets
2. The `filterB` accepts all TCP packets from the IP address `10.9.0.5` sent to port `23`
3. The `filterC` accepts all packets in the subnet `10.9.0.0/16`

Question 2 · Spoofing Packets

The objective of this task is to spoof IP packets with an arbitrary source IP address. We will spoof ICMP echo request packets and send them to another VM on the same network.

SOLUTION

Task 2A

We can also use Scapy to create packets with arbitarary data. This is called **packet spoofing**.

```
@staticmethod
def _2A():
     pkt = IP(src="10.9.0.1", dst="10.9.0.5") / ICMP()
     send(pkt, verbose=False)
```

The following code creates the simplest possible ICMP packet. Scapy allows creating packets in individual layers. Each layer is separated with a `/` where the components are initialized using constructors of the corresponding protocol type.

Here we expicitly define an ICMP packet wrapped by a custom IP packet. The other layers are automatically inferred by scapy if not provided.

The `send(...)` command sends a constructed packet via any available interface the satisfies the parameters specified in the packet.

Task 2B

More importantly, we can create packets with completely nonsensical or false information.

```
@staticmethod
def _2B():
     pkt = IP(src="1.2.3.4", dst="5.6.7.8")
     send(pkt, verbose=False)
```

Here the IP address `1.2.3.4` does not exist on the network, but is still sent via the interface successfully.

Question 3 · Traceroute

The objective of this task is to implement a simple traceroute tool using Scapy to estimate the distance, in terms of number of routers, between your VM and a selected destination.

SOLUTION

Traceroute works by manipulating the **Time To Live(TTL)** attribute of an IP packet to discover routers along a the path to a given destination one step at a time.

We start by looking at routers exactly 1 hop away, and increment TTL either until we reach our destination or there is an error. The code initialization for this is as follows.

```
def _3(target):
    ttl = 1
    while True:
        ...
```

Next we want to construct an ICMP packet that is headed to the given target. The catch is that we only give utmost as long as the value of `ttl` to live. So on the first iteration, this value is 1.

```
pkt = IP(dst=target, ttl=ttl) / ICMP()
reply = sr1(pkt, verbose=False, timeout=2)
```

Here we see a new Scapy function `sr1(...)`. This function sends a given packet once and **waits for its corresponding reply**. As a router may choose not to inform of the packet being dropped or lost, we supply a `timeout` parameter set to 2 seconds.

```
if not reply:
    print(f"{ttl:3} hops: * * *", flush=True)
    ttl += 1
    continue
```

In the case that a router did choose not to reply, we have to hope that the next router in the chain does reply. Thus we increment `ttl` and continue.

```
print(f"{ttl:3} hops: {reply[IP].src}", flush=True)
if reply[ICMP].type == 0:
    print(f"Done. Terminated at {reply[IP].src}.", flush=True)
    break
elif reply[ICMP].type == 11:
    ttl += 1
else:
    print(
        f"Stopped by ICMP Type {reply[ICMP].type} at {reply[IP].src}.",
        flush=True,
    )
    break
```

When we do get a reply, we need to test if the reply was because the packet was dropped midway, in which case we increment `ttl` and resend, or whether it successfully reached the destination when we terminate and exit.

Question 4 · Sniffing and-then Spoofing

We want to write a program that replies to all ICMP echo requests on the network regerdless of its availability

SOLUTION

This exercise can be done by reversing the source and destination of the input packet, and changing its ICMP type before sending it back. The code(which is mostly self explanatory) is shown below

```
@staticmethod
def _4():
     def spoof_ICMP(pkt):
          if IP in pkt and ICMP in pkt and pkt[ICMP].type == 8:
               print(f"Spoofed packet from {pkt[IP].src}", end=" | ")

               pkt = (
                    IP(src=pkt[IP].dst, dst=pkt[IP].src, ihl=pkt[IP].ihl)
                    / ICMP(type=0, id=pkt[ICMP].id, seq=pkt[ICMP].seq)
                    / pkt[ICMP].load
               )

               print(f"Sending {pkt.summary()}", flush=True)
               send(pkt, verbose=False)

     _ = sniff(iface=BRIDGE_INTERFACE, prn=spoof_ICMP)
```

We simply ricochet any packets we recieve that are ICMP echo requests right back with the same data as replies.

Python Code Summary

```python
from scapy.all import *
from scapy.layers.inet import ICMP, IP

BRIDGE_INTERFACE = "br-68dca9f86238"

class Task1:
    @staticmethod
    def _1A():
        def capture(pkt):
            print(pkt.summary(), flush=True)

        _ = sniff(iface=BRIDGE_INTERFACE, prn=capture)

    @staticmethod
    def _1B():
        def capture(pkt):
            print(pkt.summary(), flush=True)

        filterA = "icmp"
        filterB = "tcp and src host 10.9.0.5 and dst port 23"
        filterC = "net 10.9.0.0/16"

        _ = sniff(iface=BRIDGE_INTERFACE, filter=filterA, prn=capture)

    @staticmethod
    def _2A():
        pkt = IP(src="10.9.0.1", dst="10.9.0.5")
        send(pkt, verbose=False)

    @staticmethod
    def _2B():
        pkt = IP(src="1.2.3.4", dst="5.6.7.8")
        send(pkt, verbose=False)

    @staticmethod
    def _3(target):
        ttl = 1
        while True:
            pkt = IP(dst=target, ttl=ttl) / ICMP()
            reply = sr1(pkt, verbose=False, timeout=2)

            if not reply:
                print(f"{ttl:3} hops: * * *", flush=True)
                ttl += 1
                continue

            print(f"{ttl:3} hops: {reply[IP].src}", flush=True)
            if reply[ICMP].type == 0:
                print(f"Done. Terminated at {reply[IP].src}.", flush=True)
                break
            elif reply[ICMP].type == 11:
```

```
                ttl += 1
            else:
                print(
                    f"Stopped by unexpected ICMP Type {reply[ICMP].type}",
                    flush=True,
                )
                break

    @staticmethod
    def _4():
        def spoof_ICMP(pkt):
            if IP in pkt and ICMP in pkt and pkt[ICMP].type == 8:
                print(f"Spoofed packet from {pkt[IP].src}", end=" | ")

                pkt = (
                    IP(src=pkt[IP].dst, dst=pkt[IP].src, ihl=pkt[IP].ihl)
                    / ICMP(type=0, id=pkt[ICMP].id, seq=pkt[ICMP].seq)
                    / pkt[ICMP].load
                )

                print(f"Sending {pkt.summary()}", flush=True)
                send(pkt, verbose=False)

        _ = sniff(iface=BRIDGE_INTERFACE, prn=spoof_ICMP)
```

Setup

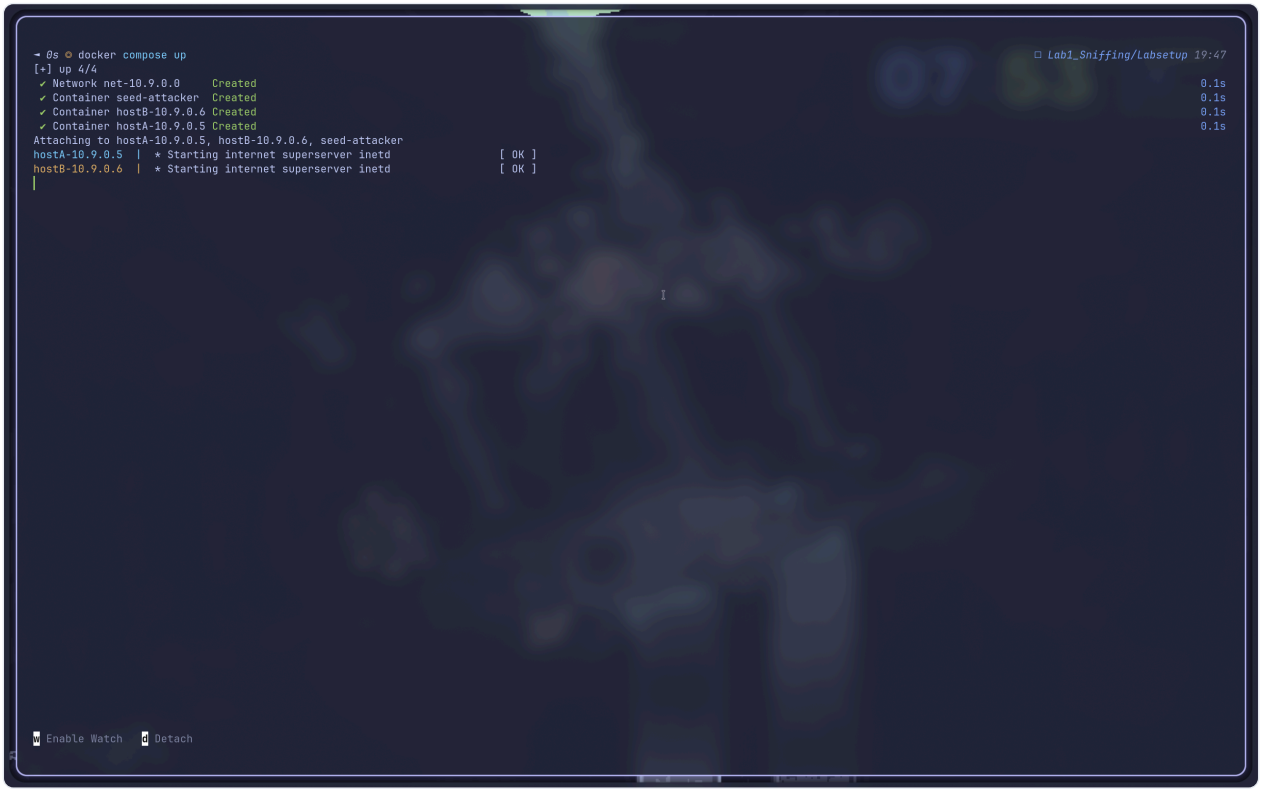

Figure 2: `docker compose up` output

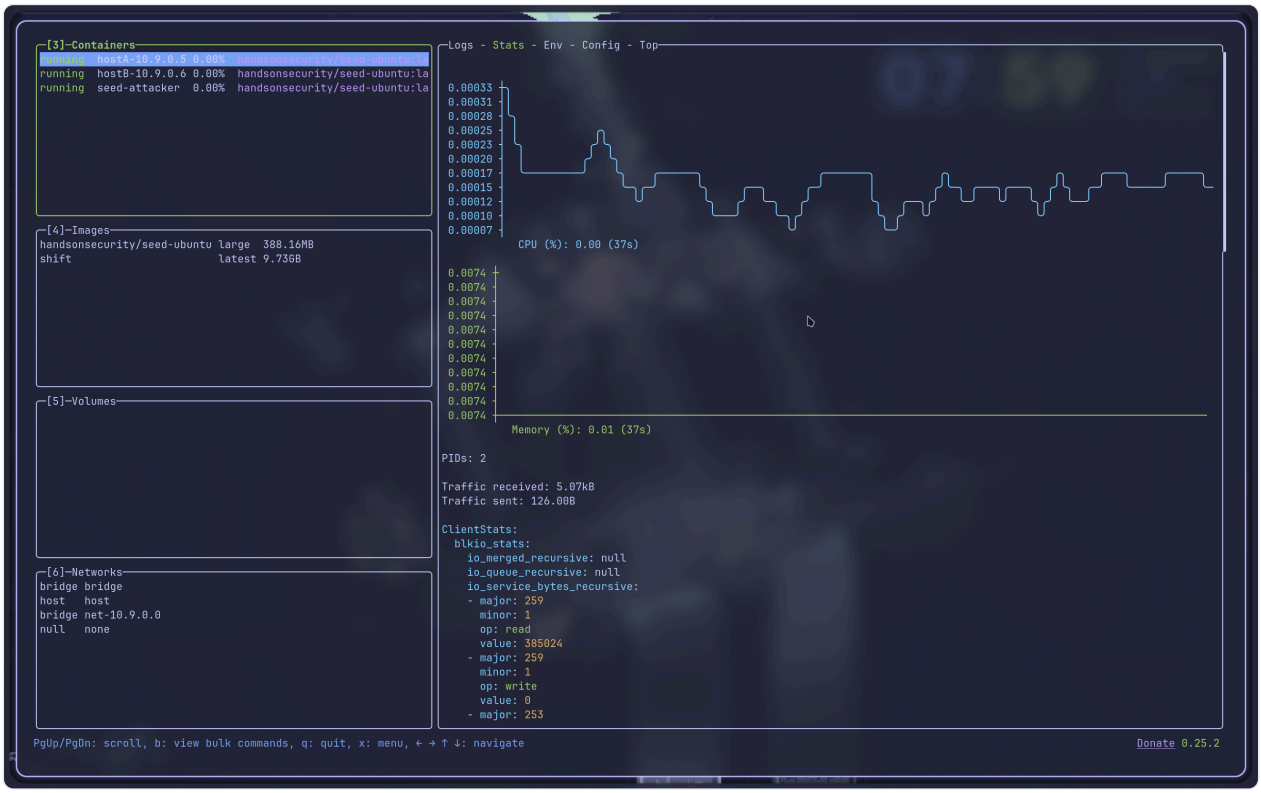

Figure 3: Running container status

Tasks

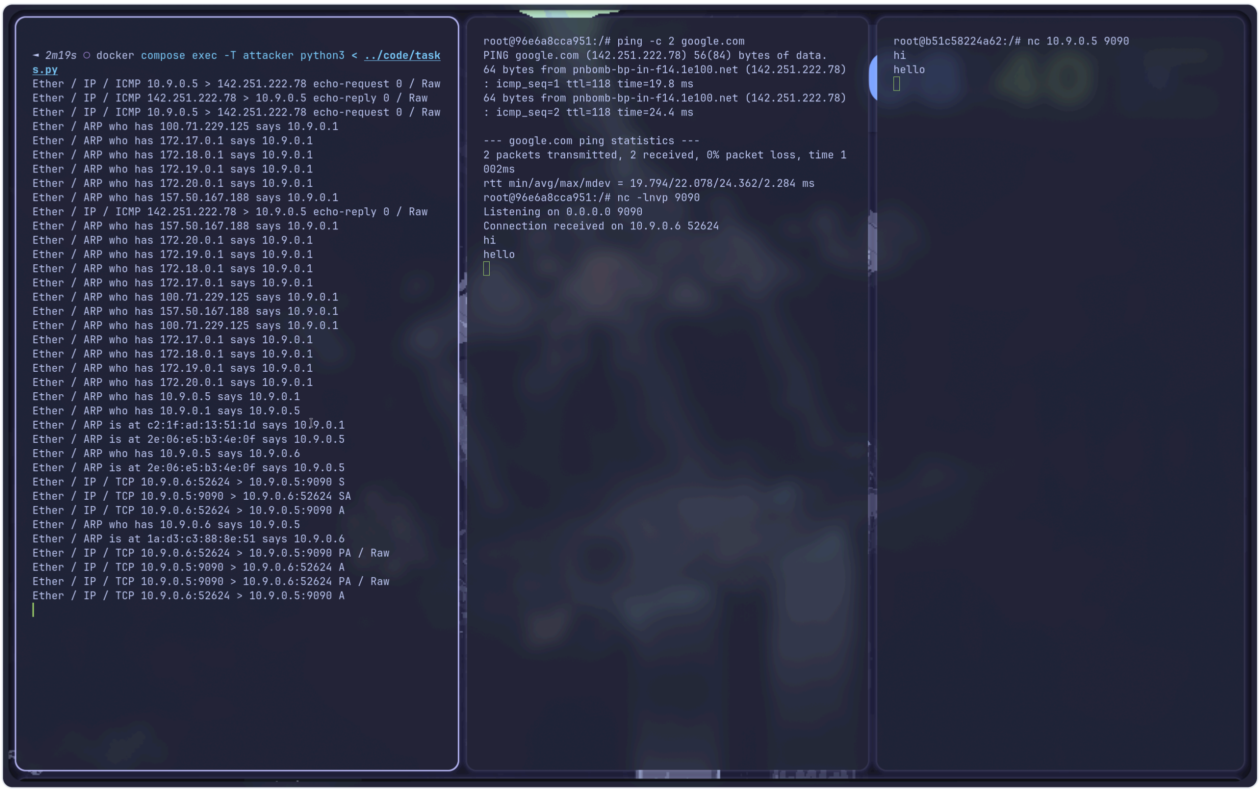

Figure 4: Task 1A

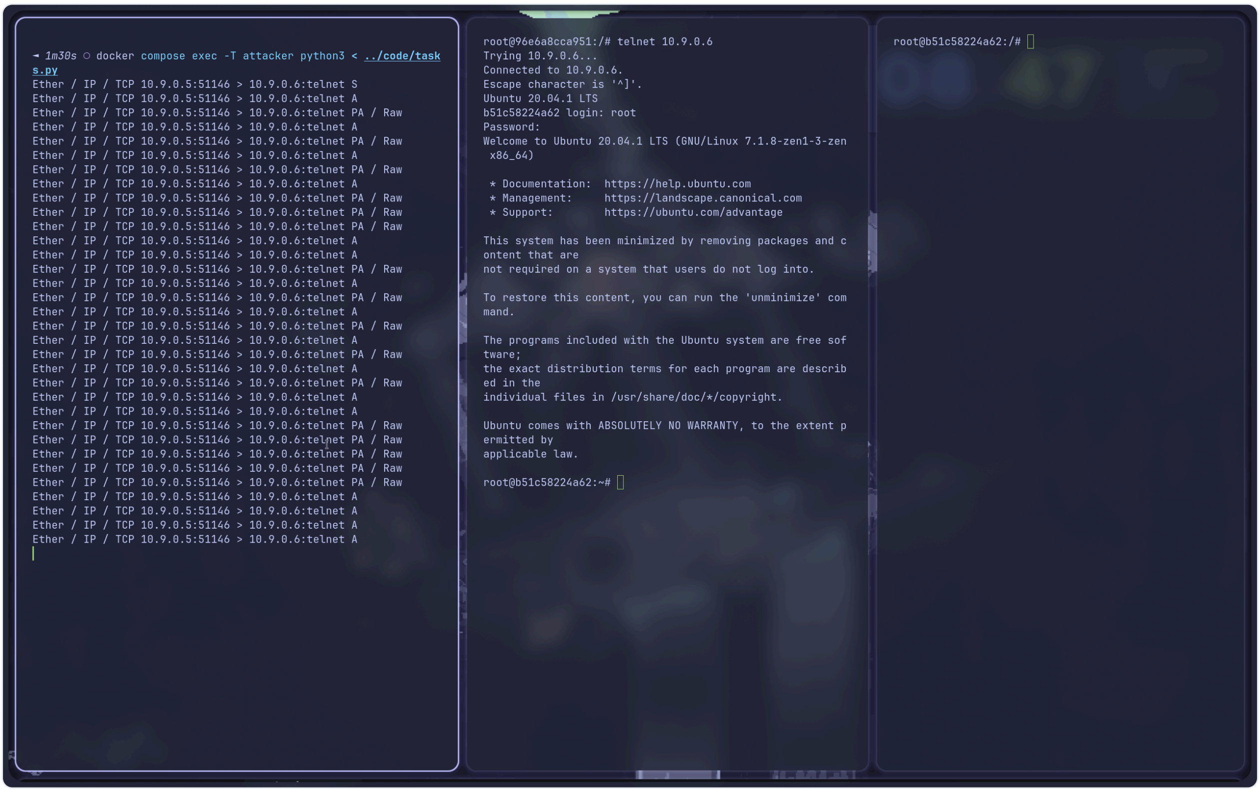

Figure 5: Task 1B

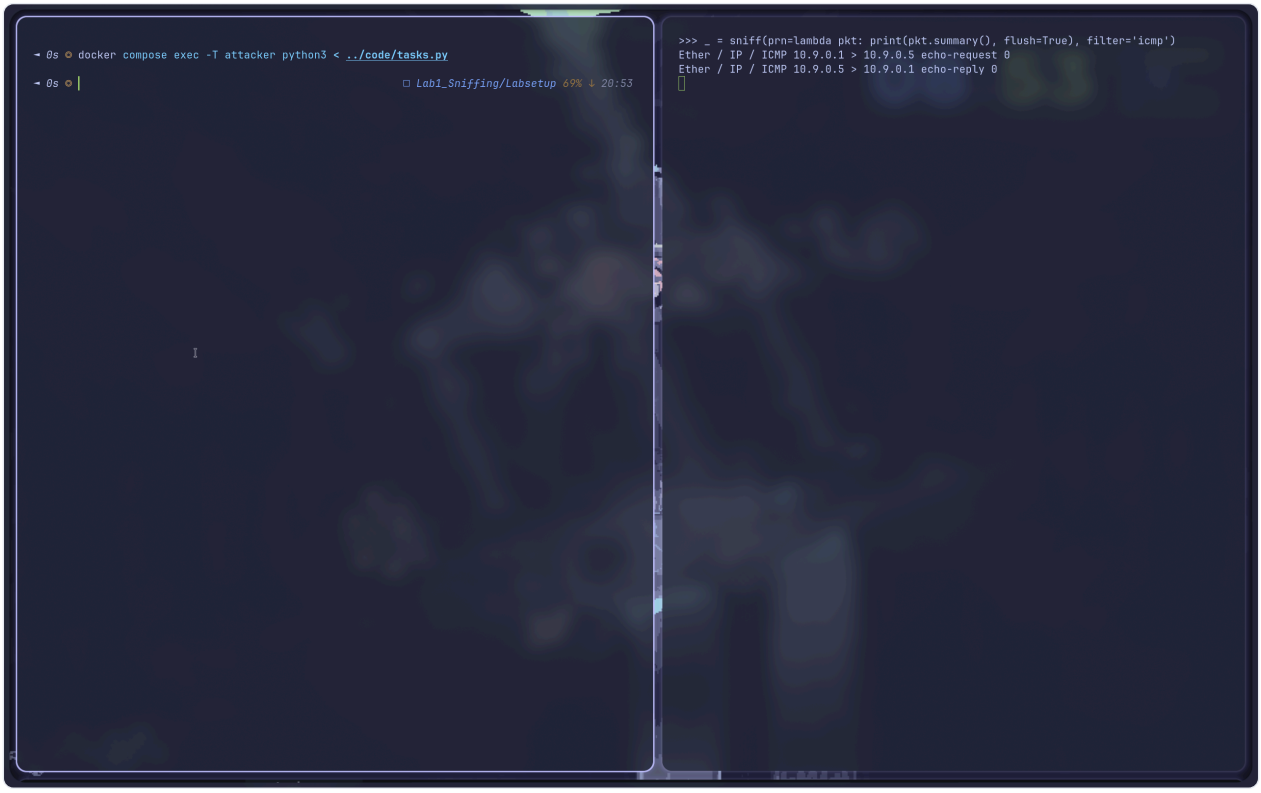

Figure 6: Task 2A

Figure 7: Task 2B

Figure 8: Task 3

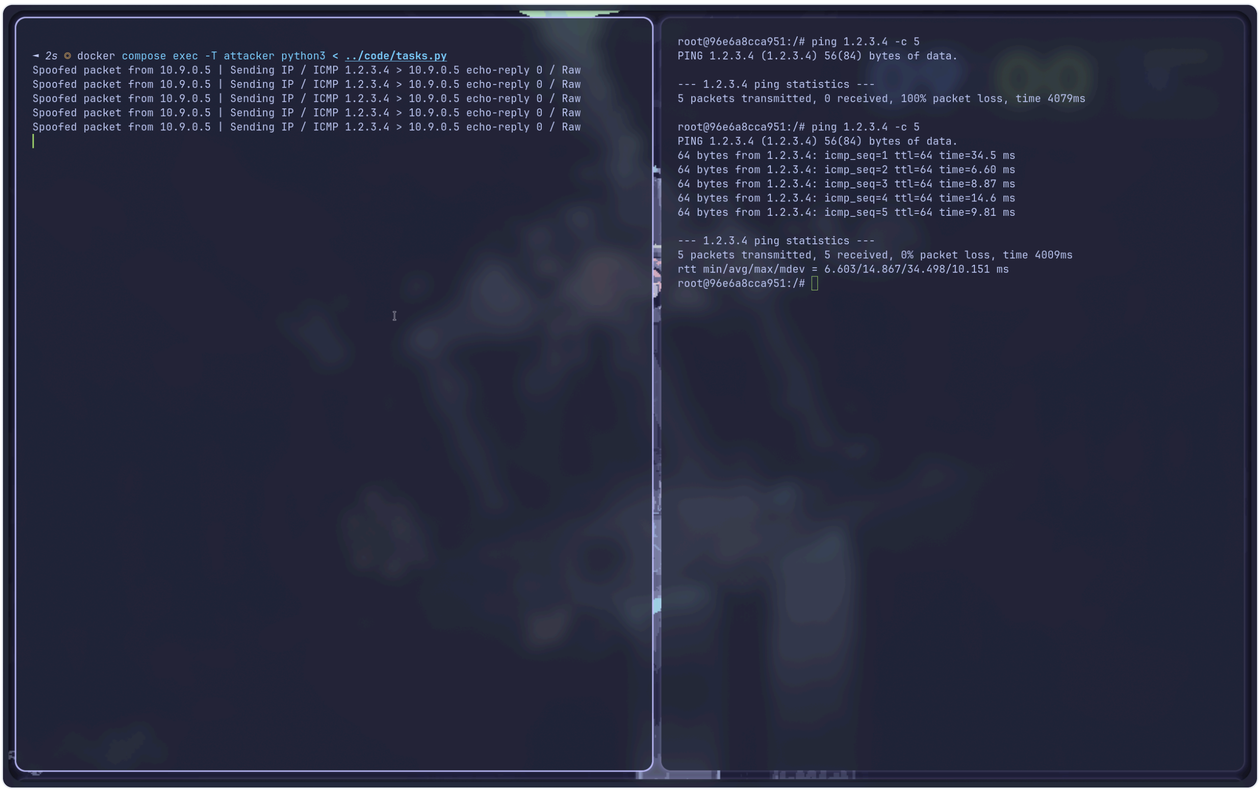

Figure 9: Task 4